Technical University of Applied Sciences Würzburg-Schweinfurt (THWS)
Faculty of Computer Science and Business Information Systems

Master Thesis

Great Research About My Favourite Topic

submitted for the completion of degree

Master of Science

in

Artificial Intelligence

Your Name

Submitted on: November 25, 2025

Supervisor: Prof. Dr. Magda Gregorova
Second Examiner: Prof. Dr. C D

Abstract (en)

A single paragraph summarizing the most important points of your thesis. This can be more than one paragraph but should not be too long. You usually write this as the very last thing of your thesis.

Abstract (de)

We are in germany so include a translation.

Acknowledgment

Here you can thank your friends who helped you throughout the studies, your parents for supporting you generally, if you wish your supervisor or any other profs and generally everybody you value.

Contents

1 Introduction

1.1 Motivation

Scalable Vector Graphics (SVG) play an essential role in modern digital design due to their scalability, compactness, and support for semantic editing. Unlike raster images, SVGs describe visual content through structured, text-based geometry using paths, shapes, and hierarchical groupings, allowing for precise control, reusability, and easy modification.

In recent years, machine learning and deep learning models have been applied to the generation of SVGs from various input modalities, such as text prompts, raster images, and sketches. Open-source research has produced a variety of approaches, including sequence models, transformer-based architectures, and differentiable optimization methods, each proposing different strategies for representing and producing vector graphics.

Given this diversity, it becomes relevant to study how these open-source models differ in their architectures, internal representations, and the structure of the SVGs they generate. Understanding these aspects can provide a clearer picture of how SVGs are represented and produced computationally, and what factors may influence the quality and editability of the resulting graphics.

1.2 Problem Statement

While multiple open-source models exist for SVG generation, they employ heterogeneous approaches in terms of architecture, data representation, and output structure. Some models treat SVGs as sequences of drawing commands, others as hierarchical objects, and still others as continuous geometric parameters optimized through differentiable rendering.

This diversity makes it challenging to compare how different approaches handle SVG generation and how these choices influence the final output. Furthermore, the evaluation

of SVG quality often depends on various factors, such as path simplicity, structural organization, and editability, which are not always measured in the same way.

Therefore, the problem addressed in this research is to systematically describe, compare, and analyze open-source SVG generation methods, focusing on their architectures, internal representations, and output characteristics, in order to understand how these design choices relate to the quality and structure of the generated SVGs.

1.3 Research Goals

1. To explain and analyze the core properties, structures, and components of SVG graphics as a foundation for understanding their representation and generation.

2. To collect, review, and categorize existing open-source methods for SVG generation across various input modalities.

3. To investigate the types of deep learning and machine learning architectures employed in SVG generation models and analyze their design principles.

4. To examine how these architectures represent SVG data internally and how their outputs are transformed into standard SVG files.

5. To explore and evaluate existing approaches, metrics, and benchmarks for assessing the quality of generated SVGs.

6. To assess and compare the output quality of current SVG generation models based on the identified evaluation criteria.

1.4 Research Questions

1. What are the main properties and structures that define an editable and high-quality SVG?

2. What open-source methods currently exist for generating SVGs from different input modalities?

3. What types of deep learning and machine learning architectures are models used for SVG generation?

4. What kinds of structure do these models produce?

5. How can the quality and editability of generated SVG be evaluated?

6. What is the observed quality of SVG outputs across different models?

7. What makes open source model suitable for testing?

8. Which open-source SVG generation models are most suitable for testing and comparison?

9. How do the selected open-source SVG generation models perform in terms of defined criteria (eg. editability, quality and practical usage)?

2 Systematic Literature Review

Kitchenham's guidelines were originally developed for evidence-based software engineering and have become one of the most widely adopted standards for conducting systematic literature reviews in computer science and technical research disciplines [1]. The methodology provides detailed, domain-specific procedures for planning, searching, selecting, and synthesizing studies in areas where research outputs are often heterogeneous—such as algorithmic models, machine learning architectures, or engineering pipelines. Unlike biomedical-focused protocols such as PRISMA [2], which were designed primarily for clinical trials and human-subject research with standardized quantitative outcomes, Kitchenham explicitly addresses challenges unique to computational research: diversity of study designs, inconsistent terminology, non-uniform evaluation metrics, and the frequent presence of code-based or algorithmic artifacts instead of comparable statistical datasets. Similarly, frameworks like SALSA [3] emphasize high-level structuring but do not prescribe the step-by-step operational procedures needed for reproducibility in technical fields.

For this thesis—situated within deep learning, machine learning, and vector-graphics generation—a Kitchenham-style SLR is needed because the literature consists of diverse model architectures, varied input modalities, and different evaluation criteria. Kitchenham's protocol supports this complexity by providing structured phases (planning, conducting, and reporting) that guide the reviewer through formulating rigorous research questions, defining search strategies, operationalizing selection criteria, assessing methodological quality, and performing narrative synthesis. This structure minimizes bias, increases reproducibility, and ensures that the review captures the full range of relevant deep-learning and vectorization approaches. Consequently, Kitchenham offers the most appropriate methodological foundation for achieving a systematic, transparent, and technically robust synthesis of current research on SVG-related generative models [4].

2.1 Phase 1 — Planning

2.1.1 Review Protocol

Databases

- arXiv
- Google Scholar
- ACM Digital Library

Search Strings/Keywords

SVG Generation

```
"SVG generation"
"scalable vector graphics" AND generation
"SVG generative models"
"vector graphics generation"
"SVG generation" AND deep learning
"SVG generation" AND machine learning
```

Raster-to-Vector Vectorization

```
"raster-to-vector" AND deep learning
"image-to-vector" AND neural network
"image vectorization" AND machine learning
"deep vectorization"
"differentiable vector graphics"
```

Text-to-SVG

```
"text-to-SVG"
"text-to-vector graphics"
```

Structure of SVG, Editability and Quality

```
"SVG Basics"
"SVG fundamentals"
"Scalable Vector Graphics"
"SVG editability"
"editable SVG"
"SVG path simplification"
"layered SVG" AND generation"
```

Survey and Review Searches

```
"SVG" AND "systematic literature review"
"vector graphics" AND survey
"vector image" AND review
"image vectorization" AND survey
"graphic generation" AND review
"SVG Generation" AND benchmark
"image vectorization" AND benchmark
```

Study Selection Criteria

Inclusion Criteria:

- **Language and Publication:** Written in English and published in a peer-reviewed journal, conference, or workshop.
- **Time Range:** Published between 2020–2025 (post-emergence of modern SVG generative models such as DeepSVG).
- **Domain Relevance:** Falls within computer vision, computer graphics, AI, machine learning, or deep learning.
- **Reproducibility:** Provides an open-source implementation or references a publicly available repository. Availability of pre-trained weights is preferred, but source code alone is sufficient if the method is reproducible.
- **SVG Output Requirement:** Describes a model, method, or architecture that directly generates SVG or structured vector outputs, including:

  - Bézier curves,
  - SVG `<path>` commands,
  - layered or hierarchical SVG structure,
  - vector strokes or SVG-like stroke sequences,
  - structured outputs suitable for editability analysis.
- **Input Modalities:** Supports at least one of:
  - image-to-SVG vectorization,
  - text-to-SVG generation,
  - multimodal SVG generation,
- **Architectural Transparency:** Provides sufficient technical detail to extract the model architecture or internal SVG representation (e.g., decoder design, transformer layers, vector tokenization, differentiable rasterization, or geometric parameterization).
- **Evaluation:** Provides quantitative metrics and/or qualitative SVG examples that allow analysis of structural quality and editability.

Exclusion Criteria:

- **No Vector Output:** The method does not generate vector or SVG outputs (pixel-only image generation).
- **Non-ML Methods:** Purely rule-based or classical algorithmic vectorization methods without machine learning or deep learning.
- **Closed-Source Systems:** No publicly available code, weights, or sufficient technical detail for reproduction.
- **Insufficient Evaluation:** Papers lacking SVG evaluation, qualitative vector examples, or any structural analysis of outputs.
- **Out-of-Scope Vector Work:** Studies focused solely on vector editing, rendering,

compression, markup optimization, or stylistic modification without SVG generation.

- **Domain Mismatch:**
  - handwriting synthesis,
  - font generation,
  - datasets,
  - sketch generation not intended for SVG graphics.
- **Temporal Exclusion:** Studies published before 2020, except when cited for background context only.
- **LLM Restriction:** Exclude proprietary LLMs that generate SVG text but do not provide architecture, training details, or reproducibility.

Study Selection Procedure

- Initial records
- Duplicate removal
- Title/abstract screening
- Full-text assessment
- Final included studies

Quality Assessment Criteria

Following the recommendations by Kitchenham for evidence-based software engineering [1, 4], each primary study must be assessed for quality, the assessment criteria must be explicitly described, applied consistently, and the overall quality of each study should be reported (e.g., high, medium, or low quality). The four Kitchenham categories used in this review are reporting quality, methodological rigor, reproducibility, and result validity. The criteria are listed below.

The following Kitchenham-style quality criteria were applied to all studies that passed the full-text screening. Each criterion was evaluated as **Yes = 1** or **No = 0**. The total score was then used to classify studies as high, medium, or low quality.

Reporting Quality

- The study states its research aim clearly.
- The model or method architecture is described in enough detail.
- The SVG/vector representation (paths, strokes, Béziers, etc.) is explained.

Methodological Rigor

- The method is appropriate for SVG/vector generation.
- The evaluation procedure is described clearly.

Reproducibility

- The study provides open-source code or enough detail to reproduce results.

Result Validity

- Quantitative metrics or qualitative SVG examples are provided and interpretable.

Scoring and Classification Each criterion was scored as either **Yes = 1** or **No = 0**. The total score determined the study's quality classification:

- **6–7 points:** High quality
- **4–5 points:** Medium quality
- **0–3 points:** Low quality (may be excluded)

Kitchenham recommends that "studies should be classified as high, medium or low quality based on quality criteria" [1].

Data Extraction Plan According to Kitchenham's guidelines for evidence-based software engineering, the purpose of data extraction is to systematically collect all information required to answer the research questions in a consistent and reproducible way. Kitchenham emphasises that a predefined data extraction form must be created *during the planning phase* to ensure uniformity and reduce subjective variation among reviewers [1].

Predefined Data Extraction Form Kitchenham states that researchers should develop a structured data extraction form before conducting the review. This prevents ad-hoc decisions and ensures that all necessary information is collected consistently across all studies [1].

Alignment with Research Questions Kitchenham notes that extracted data must be driven directly by the review's research questions. The form should therefore include only fields that support answering the research questions or performing the planned synthesis [1].

Information to Extract The guidelines specify that extraction should capture both bibliographic information and structured study attributes, including study context, aims, methodology, data used, experimental setup, and results. Kitchenham emphasises that the level of detail must be sufficient for later comparison and synthesis [1].

Reviewer Consistency Kitchenham recommends that extracted data should be validated—ideally by a second reviewer—to minimise bias and ensure consistency. If this is not possible, reviewers should re-check their own extraction for consistency [1].

Structured (Not Free-Form) Fields Kitchenham advises the use of structured or semi-structured fields to minimise subjectivity and ensure that comparable information is extracted across studies. Examples include fixed fields for model architecture, datasets, evaluation metrics, and SVG output structure [1].

Avoiding Unnecessary Information Only data needed for answering the research questions or for synthesis should be extracted. Kitchenham warns that recording irrelevant details increases effort and contributes no value to the final review [4].

Ensuring Comparability Across Studies Finally, Kitchenham highlights that the extraction process must ensure that the information gathered from different studies is comparable and suitable for aggregation or narrative synthesis [4]. Using a consistent extraction form enables systematic comparison of SVG generation models.

Table 2.1: Data Extraction Form for Included Studies

Field	Extracted Information
Bibliographic Information Title Authors Year Venue	
Study Aim (RQ1)	What the paper aims to achieve; problem being solved; contribution.
Input Modality (RQ2)	Image-to-SVG, Text-to-SVG, Multimodal, Sketch-to-SVG, or other.
Model Architecture (RQ3)	Core model type (Transformer, Diffusion, VAE, autoregressive decoder, DiT, implicit model, VLM/LLM, etc.); training strategy; important modules; optimization method.
Internal SVG Representation (RQ4)	How the model represents vector graphics internally: paths, Bézier curves, control points, strokes, layers, hierarchy, tokens, neural implicit representation, etc.
SVG Output Structure (RQ4)	Description of the output SVG structure: number of shapes, path types, hierarchy, style constraints, editability features.
Evaluation Metrics (RQ5)	Quantitative metrics (e.g., FID, CLIP, PSNR, LPIPS, SSIM, MSE, R-Precision); Qualitative evaluation (figures, examples).
Editability Assessment (RQ1 & RQ5)	Any information on path simplicity, layer structure, number of Bézier curves, redundancy, cleanliness, or user-editability.
Results Summary (RQ6)	Key results: performance vs baselines, strengths, weaknesses, notable findings.
Open-Source Availability (RQ7)	Code URL; availability of pretrained weights; reproducibility details.
Suitability for Testing (RQ8)	Is this model appropriate for downstream testing and comparison? Why?
Additional Notes	Any extra information relevant for synthesis.

2.2 Phase 2 — Conducting the Review

2.2.1 Execution of the Search Strategy

The search strings defined in Phase 1 were executed across the selected databases (Google Scholar, arXiv, and ACM Digital Library) to identify relevant literature on SVG generation and vector-based deep learning methods. Searches were performed using the conceptual string groups defined in the search strategy, with database-specific syntax adjustments where necessary. All records retrieved by these searches were exported and documented before any filtering or removal of duplicates. Table 2.2 summarises the number of studies returned by each search group and database.

Table 2.2: Search Log Summary Across Databases and Conceptual Groups

Search Group	**Google Scholar**	**arXiv**	**ACM DL**	**Total**
SVG Generation	18	15	4	37
Raster-to-Vector	10	8	2	20
Text-to-SVG / Multimodal	7	10	5	22
Surveys/Benchmarks/Reviews	6	3	1	10
SVG Fundamentals / Background	3	3	2	8
Total Hits	**44**	**39**	**12**	**95**

2.2.2 Removing of Duplications

After merging all search results, duplicate records—including repeated titles and preprint–conference duplicates—were removed. This resulted in 72 unique studies.

2.2.3 Study Selection

- Stage 1: Title, Keywords and Abstract screening.

  After removing duplicates, **72** unique studies remained. A brief title and abstract screening excluded works clearly unrelated to SVG generation (e.g., sketch-only models, raster methods, classical vectorization, or closed-source LLMs), resulting in **37** papers proceeding to full-text review.

- Stage 2: Full-text screening

A full-text assessment was conducted on the **37** studies that passed the title and abstract screening. Each paper was examined for reproducibility, architectural transparency, and whether it genuinely produced SVG or structured vector outputs as defined by the inclusion criteria. In total, **10** studies were excluded at this stage: four lacked any open-source implementation, two were benchmark papers evaluating proprietary LLMs such as GPT, Gemini, or Claude, two used non-reproducible LLM-based pipelines without sufficient methodological detail, and one relied on a purely rule-based vectorization approach. Following this eligibility check, **27** studies remained for quality assessment.

Table 2.3: Reasons for Exclusion During Full-Text Screening

Exclusion Reason	**Count**
No open-source implementation	4
Proprietary LLM benchmark studies (GPT, Gemini, Claude)	2
Non-reproducible LLM-based pipeline	2
Purely rule-based vectorization method	2
Total Excluded	10

Table 2.4: Study Selection Summary Across Screening Stages

Selection Stage	**Number of Studies**
Initial search results (all databases)	95
After duplicate removal	72
After title and abstract screening	37
After full-text screening	27

Table 2.5: Data Extraction Summary for NeuralSVG (2025)

Field	Extracted Information
Title	NeuralSVG: An Implicit Representation for Text-to-Vector Generation
Authors	Sagi Polaczek, Yuval Alaluf, Elad Richardson, Yael Vinker, Daniel Cohen-Or
Year	2025
Venue	Not reported
Study Aim (RQ1)	NeuralSVG introduces an implicit neural representation for generating vector graphics from text. It aims to produce ordered, layered, simple, and editable SVGs, addressing issues of over-parameterization and disorganized structures in previous methods.
Input Modality (RQ2)	Text-to-SVG
Model Architecture (RQ3)	Implicit neural representation encoded in MLP weights. Uses positional encoding and two MLP branches (control points and color). Optimized using Score Distillation Sampling (SDS) guided by Stable Diffusion. Dropout regularization encourages meaningful layered structure.
Internal SVG Representation (RQ4)	Each index i maps to a shape z_i defined by control points p_i and fill color c_i. All shapes are produced from the implicit function f_θ.
SVG Output Structure (RQ4)	Typically 16 shapes in a layered, ordered hierarchy. Each shape is a closed Bézier form with four cubic curves (12 control points plus a fill color). Outputs are designed for human editability.
Evaluation Metrics (RQ5)	CLIP cosine similarity, R-Precision (R-Prec), and cumulative CLIP similarity (for ordering).
Editability Assessment (RQ1 & RQ5)	Produces semantically meaningful, standalone shapes with low redundancy, enabling practical editing.
Results Summary (RQ6)	Outperforms VectorFusion and SVGDreamer under the same 16-shape constraint. Baselines require 4–16× more shapes for comparable fidelity. NeuralSVG learns meaningful ordering: early shapes capture core semantics.
Open-Source Availability (RQ7)	Project page: `https://sagipolaczek.github.io/NeuralSVG/`
Suitability for Testing (RQ8 & RQ9)	Yes. Evaluated on the standard 128-prompt VectorFusion benchmark with strong quantitative and qualitative comparisons. Suitable for downstream tests.
Additional Notes	Implicit representation allows inference-time control (e.g., colors, aspect ratio) and can produce vector sketches at multiple abstraction levels.

2.2.4 8. Data Synthesis

8.1 Narrative Synthesis Summarize findings, patterns, and categories.

8.2 Sensitivity Analysis Remove low-quality studies and reassess.

2.3 Phase 3 — Reporting

2.3.1 9. Prepare the SLR Report

Document all methodology, results, synthesis, and identified gaps.

Bibliography

[1] Barbara Kitchenham. *Procedures for Performing Systematic Reviews*. en. Technical Report. Keele University and NICTA, 2004.

[2] David Moher, Alessandro Liberati, Jennifer Tetzlaff, Douglas G. Altman. "Preferred reporting items for systematic reviews and meta-analyses: the PRISMA statement". en. In: *BMJ* 339 (July 2009). Publisher: British Medical Journal Publishing Group Section: Research Methods & Reporting, b2535.

[3] *A typology of reviews: an analysis of 14 review types and associated methodologies - Grant - 2009 - Health Information & Libraries Journal - Wiley Online Library*.

[4] Barbara Kitchenham, O. Pearl Brereton, David Budgen, Mark Turner, John Bailey, Stephen Linkman. "Systematic literature reviews in software engineering – A systematic literature review". en. In: *Information and Software Technology* 51.1 (Jan. 2009), pp. 7–15.

List of Figures

Appendix

Great table.

Headline 1	**Headline 2**
Titel 1	Description 1
Titel 2	Description 2

Table 1: Overview in a great table.

Declaration on oath

I hereby certify that I have written my master thesis independently and have not yet submitted it for examination purposes elsewhere. All sources and aids used are listed, literal and meaningful quotations have been marked as such.

November 25, 2025, Your Name

Consent to plagiarism check

I hereby agree that my submitted work may be sent to PlagAware (https://my.plagaware.com/) in digital form for the purpose of checking for plagiarism and that it may be temporarily (max. 5 years) stored in the database maintained by PlagScan as well as personal data which are part of this work may be stored there.

Consent is voluntary. Without this consent, the plagiarism check cannot be prevented by removing all personal data and protecting the copyright requirements. Consent to the storage and use of personal data may be revoked at any time by notifying the faculty.

November 25, 2025, Your Name